ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 7 - Due date 03/06/26

Yeeun Kim

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github. And to do so you will need to fork our repository and link it to your RStudio.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A07_Sp26.Rmd"). Then change "Student Name" on line 4 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Sakai.

Packages needed for this assignment: "forecast","tseries". Do not forget to load them before running your script, since they are NOT default packages.\

Set up

```r
#Load/install required package here
library(lubridate)
```

```
##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```r
library(ggplot2)
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.5.2

## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
library(Kendall)
```

```
## Warning: package 'Kendall' was built under R version 4.5.2
```

```r
library(tseries)
```

```
## Warning: package 'tseries' was built under R version 4.5.2
```

```r
library(outliers)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr   1.1.4     v stringr 1.5.2
## v forcats 1.0.0     v tibble  3.3.0
## v purrr   1.1.0     v tidyr   1.3.1
## v readr   2.1.5

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##     stamp
```

Importing and processing the data set

Consider the data from the file "Net_generation_United_States_all_sectors_monthly.csv". The data corresponds to the monthly net generation from January 2001 to December 2020 by source and is provided by the US Energy Information and Administration. **You will work with the natural gas column only**.

Q1

Import the csv file and create a time series object for natural gas. Make you sure you specify the **start=** and **frequency=** arguments. Plot the time series over time, ACF and PACF.

```r
#Importing time series data from text file#
net_gen <- read.csv(file="./Data/Net_generation_United_States_all_sectors_monthly.csv",header=TRUE,skip=
ng_gen <- net_gen %>%
  select(1,4)

#Preparing the data - create date object and rename columns
ng_gen_processed <-
```

```r
  ng_gen %>%
  mutate( Month = my(Month) ) %>%
  rename( NG_Generation = natural.gas.thousand.megawatthours ) %>%
  arrange( Month )

head(ng_gen_processed)
```

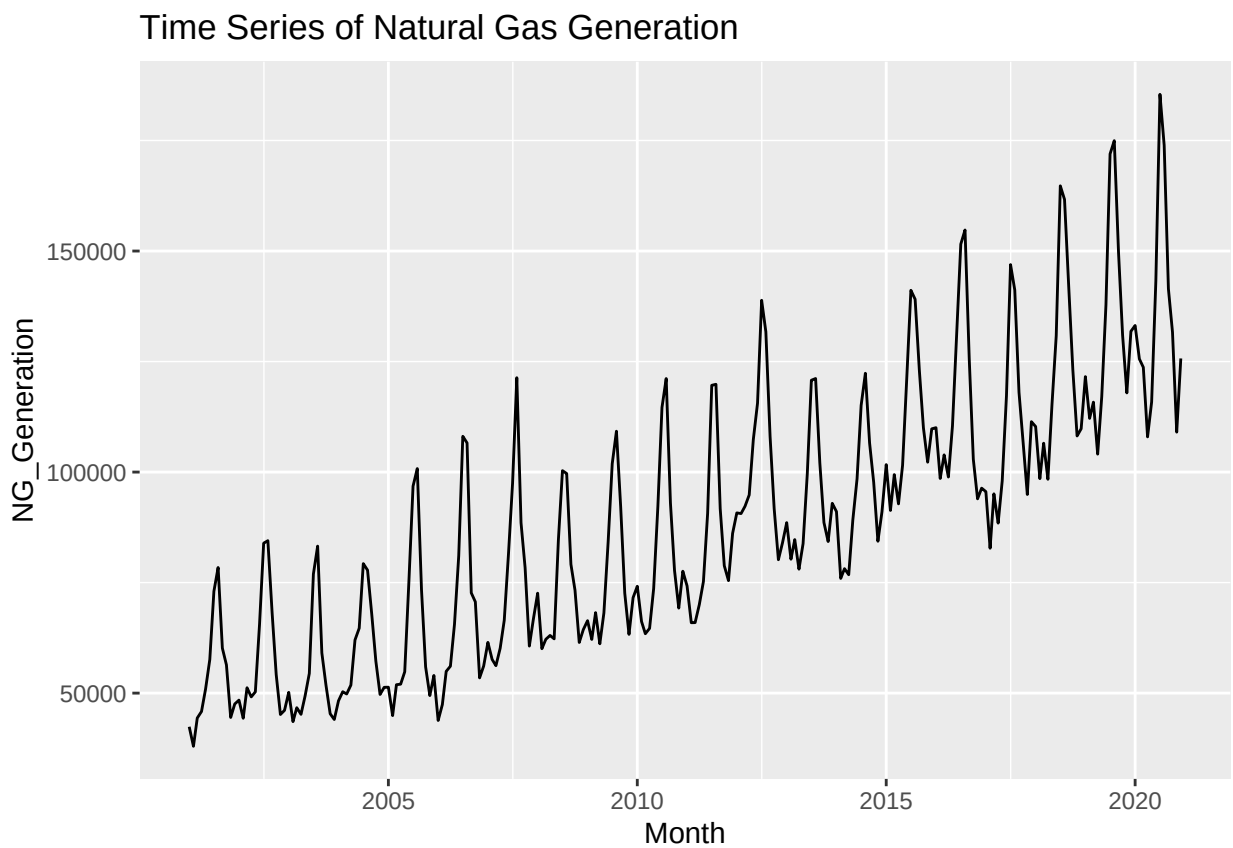
```
##        Month NG_Generation
## 1 2001-01-01      42388.66
## 2 2001-02-01      37966.93
## 3 2001-03-01      44364.41
## 4 2001-04-01      45842.75
## 5 2001-05-01      50934.21
## 6 2001-06-01      57603.15
```

```r
summary(ng_gen_processed)
```

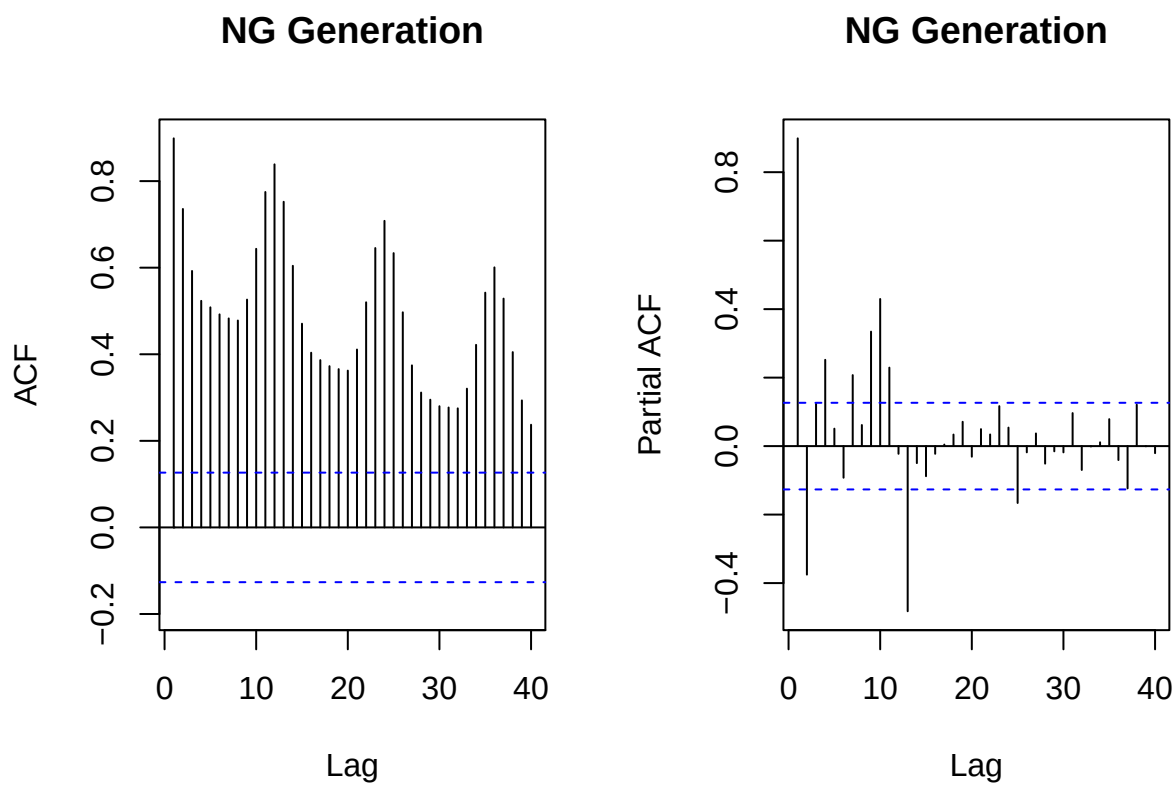
```
##      Month            NG_Generation
##  Min.   :2001-01-01   Min.   : 37967
##  1st Qu.:2005-12-24   1st Qu.: 62245
##  Median :2010-12-16   Median : 84415
##  Mean   :2010-12-16   Mean   : 88028
##  3rd Qu.:2015-12-08   3rd Qu.:108385
##  Max.   :2020-12-01   Max.   :185445
```

```r
#Time series object
ts_ng_gen <- ts(ng_gen_processed[,2],start = c(2001,1), frequency = 12)

#Plots
TS_Plot <-
  ggplot(ng_gen_processed, aes(x=Month, y=NG_Generation)) +
      geom_line() +
  ggtitle("Time Series of Natural Gas Generation")
plot(TS_Plot)
```




```
#ACF and PACF plots
par(mfrow=c(1,2))
ACF_Plot <- Acf(ng_gen_processed$NG_Generation, lag = 40, plot = TRUE, main = "NG Generation")
PACF_Plot <- Pacf(ng_gen_processed$NG_Generation, lag = 40, main = "NG Generation")
```



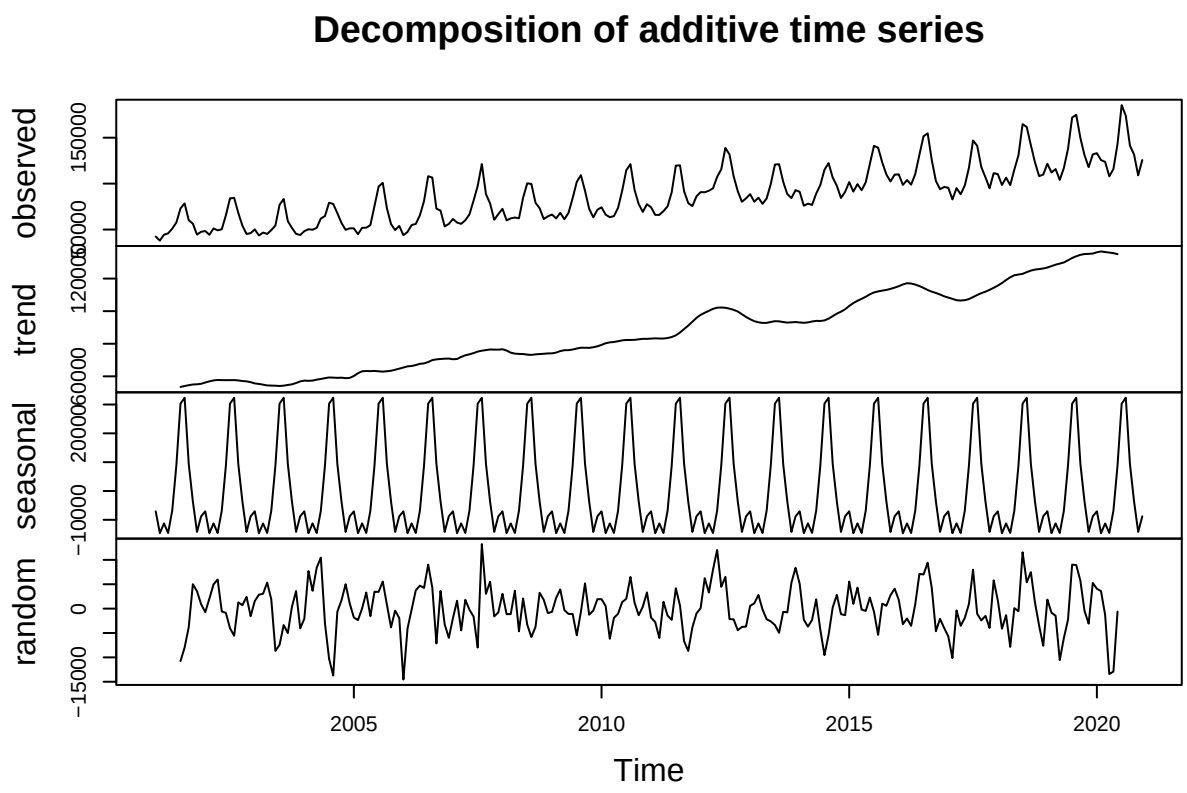

```
par(mfrow=c(1,1))
```

Q2

Using the *decompose*() and the *seasadj*() functions create a series without the seasonal component, i.e., a deseasonalized natural gas series. Plot the deseasonalized series over time and corresponding ACF and PACF. Compare with the plots obtained in Q1.

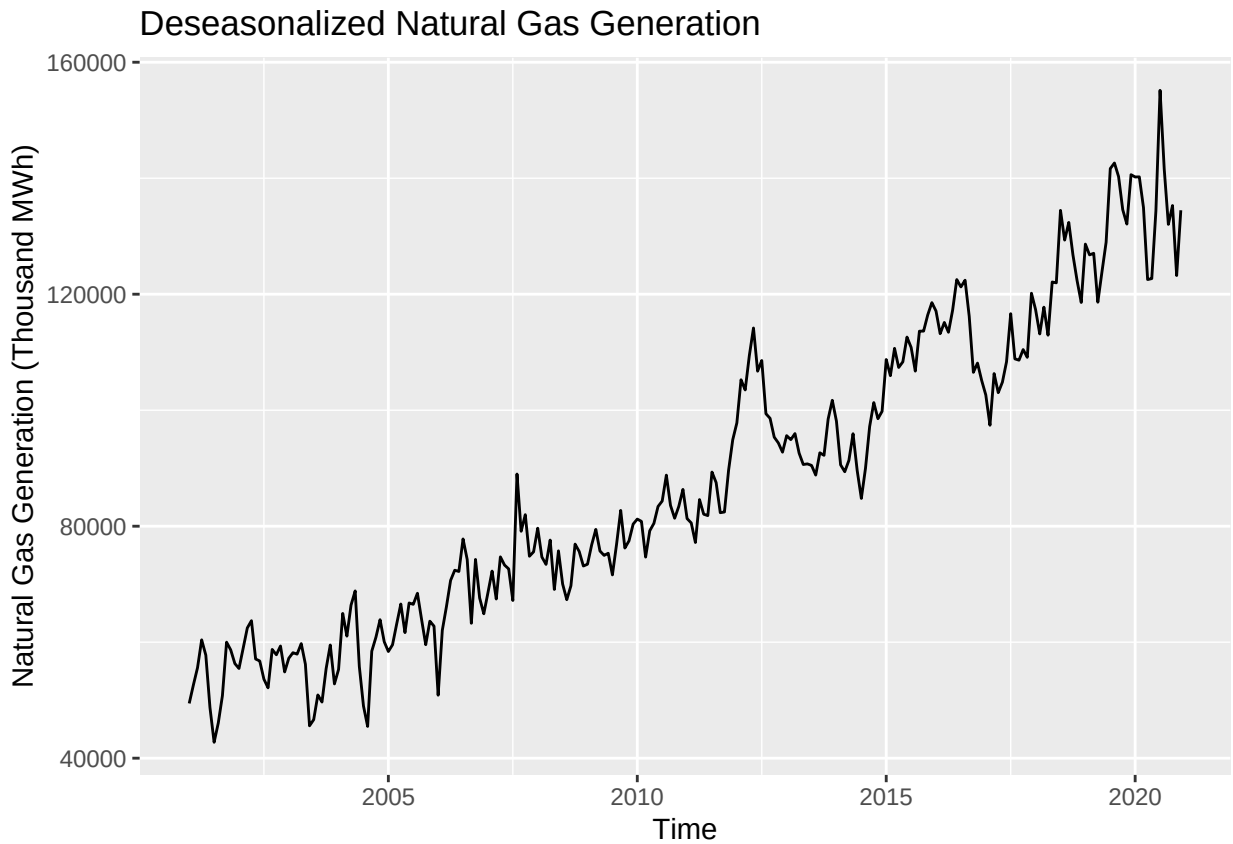
> Answer: In the original series, the ACF has spikes at lags 12, 24, and 36, showing seasonality, while the PACF shows spikes at lag 1 and lag 12, suggesting non-seasonal and seasonal AR components. In the Q2 plots, the lags at ACF and PACF disappear, indicating the seasonal component has been removed. However, the deseasonalized ACF starts near 1.0 and decays slowly, which indicates a non-stationarity and a trend component. The PACF shows a spike at lag 1 and a cut-off.

```
#Decompose the time series
decomp_ng <- decompose(ts_ng_gen, type = "additive")
plot(decomp_ng)
```

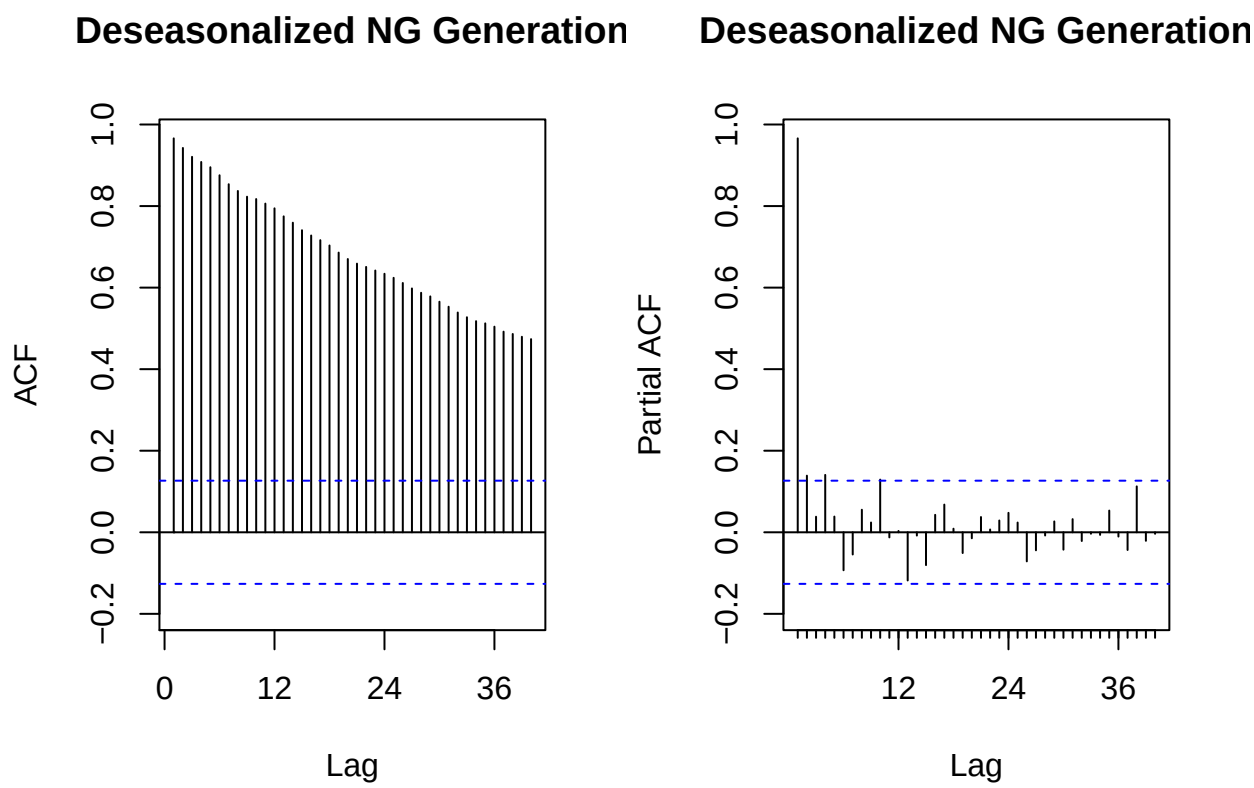



```r
#Remove seasonal component
deseason_ng <- seasadj(decomp_ng)

#Plot
autoplot(deseason_ng) +
  ggtitle("Deseasonalized Natural Gas Generation") +
  xlab("Time") +
  ylab("Natural Gas Generation (Thousand MWh)")
```




```
#ACF and PACF plots
par(mfrow=c(1,2))
Acf(deseason_ng, lag = 40, plot = TRUE, main = "Deseasonalized NG Generation")
Pacf(deseason_ng, lag = 40, main = "Deseasonalized NG Generation")
```




```r
par(mfrow=c(1,1))
```

Modeling the seasonally adjusted (i.e., the deseasonalized series)

Q3

Run the ADF test and Mann Kendall test on the deseasonalized data from Q2. Report and explain the results.

> Answer: The ADF test shows a p-value of 0.01, suggesting that we reject the null hypothesis, which means there is no stochastic trend and the deseasonalized series is stationary. The Mann-Kendall shows a tau value or 0.843 and a p-value of less than 2.22e-16, indicating that there is a deterministic trend in the series. According to the Mann-Kendall result and the slowly decaying ACF in Q2, we can say a trend reamins in the deseasonalized series. Thus, differencing the series is needed to remove the trend.

```r
# ADF Test
adf_test <- adf.test(deseason_ng, alternative = "stationary")
```

```
## Warning in adf.test(deseason_ng, alternative = "stationary"): p-value smaller
## than printed p-value
```

```r
print(adf_test)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  deseason_ng
## Dickey-Fuller = -4.0271, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
# Mann-Kendall Test
mk_test <- MannKendall(deseason_ng)
summary(mk_test)
```

```
## Score =  24186 , Var(Score) = 1545533
## denominator =  28680
## tau = 0.843, 2-sided pvalue =< 2.22e-16
```

Q4

Using the plots from Q2 and test results from Q3 identify the ARIMA model parameters p, d and q. Note that in this case because you removed the seasonal component prior to identifying the model you don't need to worry about seasonal component. Clearly state your criteria and any additional function in R you might use. DO NOT use the *auto.arima*() function. You will be evaluated on ability to understand the ACF/PACF plots and interpret the test results.

> Answer: Since the ACF in Q2 decays slowly and the Mann-Kendall test shows an upward trend, first-order differencing was applied, which means d=1. Based on the ACF and PACF plots, p and q could be either 0 or 1. So, I have three candidate models, and I compared the AIC vaule to verify which model is appropriate. As a result, model_111 has the lowest AIC value among three candidates. Thus, p=1, d=1, and q=1.

```r
#Differencing
deseason_ng_diff <- diff(deseason_ng, differences = 1)

#Check stationarity
adf.test(deseason_ng_diff)
```

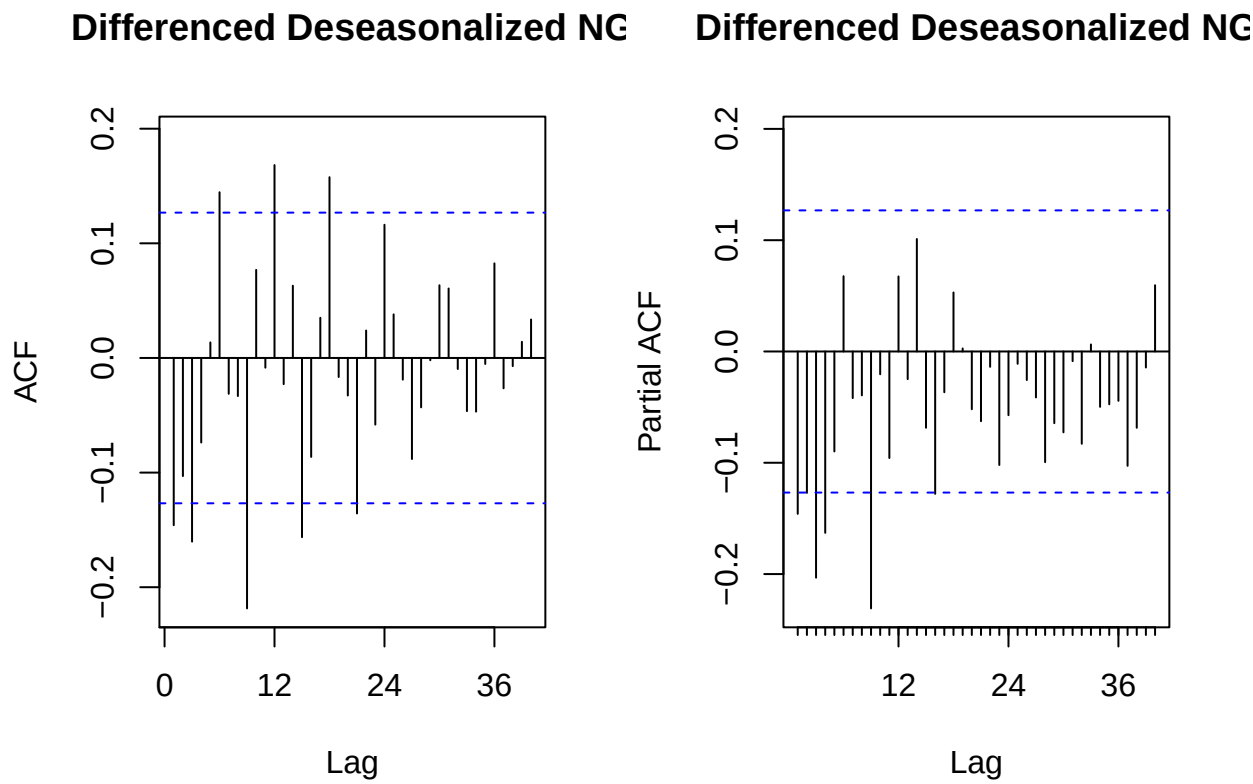
```
## Warning in adf.test(deseason_ng_diff): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  deseason_ng_diff
## Dickey-Fuller = -6.9137, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
MannKendall(deseason_ng_diff)
```

```
## tau = -0.0105, 2-sided pvalue =0.80939
```

```r
#The ACF and PACF plots affter differencing
par(mfrow=c(1,2))
Acf(deseason_ng_diff, lag=40, main="Differenced Deseasonalized NG")
Pacf(deseason_ng_diff, lag=40, main="Differenced Deseasonalized NG")
```




```r
par(mfrow=c(1,1))

#Candidates
model_011 <- Arima(deseason_ng, order=c(0,1,1))
model_012 <- Arima(deseason_ng, order=c(0,1,2))
model_111 <- Arima(deseason_ng, order=c(1,1,1))

#AIC values
AIC(model_011, model_012, model_111)
```

```
##           df      AIC
## model_011  2 4796.239
## model_012  3 4789.286
## model_111  3 4784.955
```

Q5

Use `Arima()` from package "forecast" to fit an ARIMA model to your series considering the order estimated in Q4. You should allow constants in the model, i.e., `include.mean = TRUE` or `include.drift=TRUE`. **Print the coefficients** in your report. Hint: use the `cat()` or `print()` function to print.

```r
# Fit ARIMA(1,1,1) with drift
model_111_drift <- Arima(deseason_ng, order = c(1,1,1), include.drift = TRUE)
summary(model_111_drift)
```

```
## Series: deseason_ng
## ARIMA(1,1,1) with drift
##
## Coefficients:
##          ar1      ma1     drift
##       0.7065  -0.9795  359.5052
## s.e.  0.0633   0.0326   29.5277
##
## sigma^2 = 26980609:  log likelihood = -2383.11
## AIC=4774.21   AICc=4774.38   BIC=4788.12
##
## Training set error measures:
##                     ME     RMSE     MAE        MPE     MAPE      MASE
## Training set -141.3123 5150.819 3984.38 -0.7171368 4.850437 0.4871225
##                      ACF1
## Training set -0.003014461
```

```r
# Print coefficients
cat("ARIMA(1,1,1) Model Coefficients:\n")
```

```
## ARIMA(1,1,1) Model Coefficients:
```

```r
print(model_111_drift$coef)
```

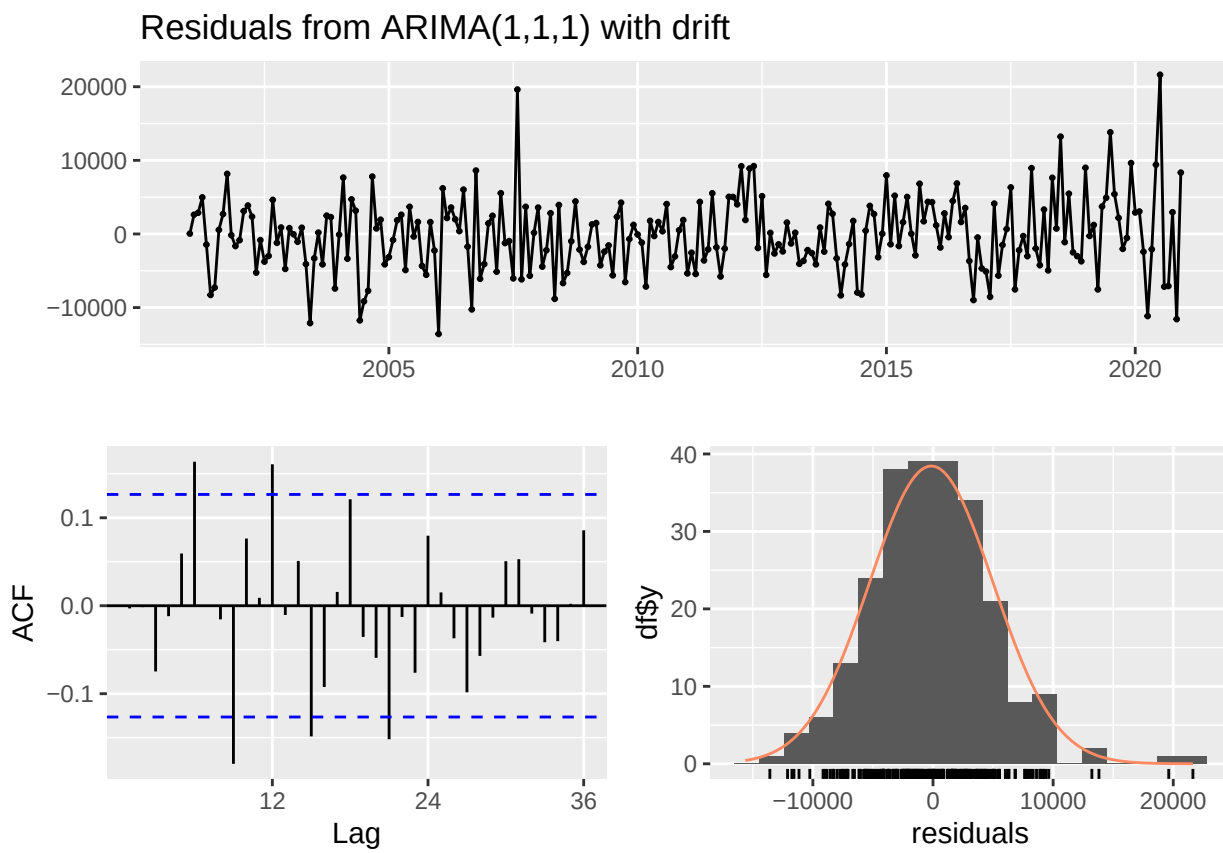
```
##         ar1         ma1       drift
##   0.7065237  -0.9794655 359.5051902
```

Q6

Now plot the residuals of the ARIMA fit from Q5 along with residuals ACF on the same window. You may use the *checkresiduals*() function to get the plots. Do the residual series look like a white noise series? Why?

> Answer: The residuals series don't look like a white noise series. The residual ACF shows a spike at lag 12, indicating that some seasonal component remains. Also, there are outliers around 2007 and 2021.

```r
checkresiduals(model_111_drift)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1) with drift
## Q* = 48.356, df = 22, p-value = 0.0009736
##
## Model df: 2.   Total lags used: 24
```

Modeling the original series (with seasonality)

Q7

Repeat Q3-Q6 for the original series (the complete series that has the seasonal component). Note that when you model the seasonal series, you need to specify the seasonal part of the ARIMA model as well, i.e., P, D and Q.

> Answer: The ADF test on the original series shows a p-value of 0.01, indicating no stochastic trend. The Mann-Kendall test (tau=0.651, p<2.22e-16) tells a deterministic upward trend. To achieve stationarity, seasonal differencing (D=1) was applied, which reduced the Mann-Kendall tau from 0.651 to 0.0965. First-order differencing (d=1) was also applied, turning the tau to -0.00409, and p-value of 0.928, confirming stationarity. From the ACF and PACF of the double differenced series, a negative spike at lag 1 suggested q=1 and the PACF indicated p=1. At the seasonal lags, the negative spike at lag 12 in the ACF with no spike at lag 24 suggested Q=1 and P=0. After comparing AIC values, ARIMA(1,1,1)(0,1,1)12 has the lowest AIC of 4552.39.

The coefficients are ar1=0.7323, ma1=-0.9819, and sma1=-0.7017. The residual ACF showed no spikes at any lag, and the residuals fluctuated randomly around zero, indicating that the model captured the series structure.

```r
#ADF Test
adf.test(ts_ng_gen, alternative = "stationary")
```

```
## Warning in adf.test(ts_ng_gen, alternative = "stationary"): p-value smaller
## than printed p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  ts_ng_gen
## Dickey-Fuller = -8.9602, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
#Mann-Kendall Test
MannKendall(ts_ng_gen)
```

```
## tau = 0.651, 2-sided pvalue =< 2.22e-16
```

```r
#Seasonal differencing
ts_ng_seasonal_diff <- diff(ts_ng_gen, lag = 12)

#ADF and MK after the seasonal differencing
adf.test(ts_ng_seasonal_diff)
```

```
## Warning in adf.test(ts_ng_seasonal_diff): p-value smaller than printed p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  ts_ng_seasonal_diff
## Dickey-Fuller = -4.6483, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
MannKendall(ts_ng_seasonal_diff)
```

```
## tau = 0.0965, 2-sided pvalue =0.030096
```

```r
#First differencing
ts_ng_diff <- diff(ts_ng_seasonal_diff, differences = 1)
adf.test(ts_ng_diff)
```

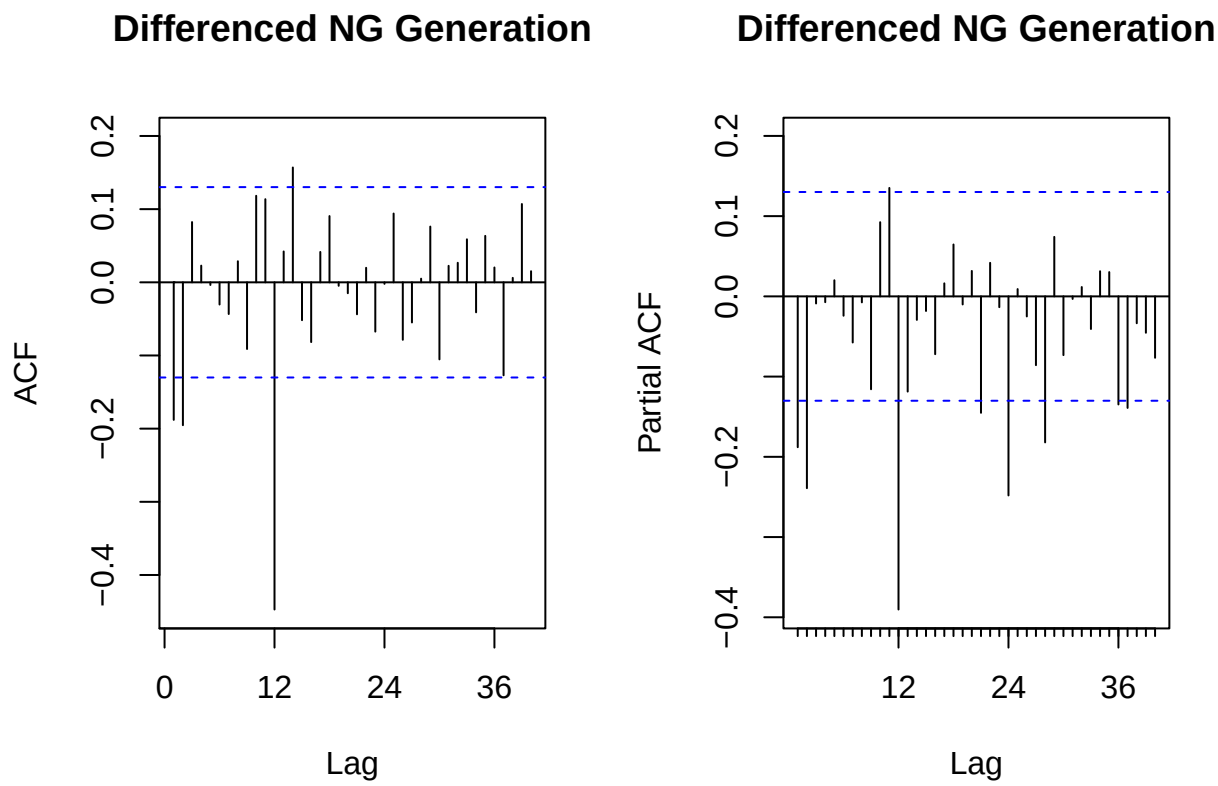
```
## Warning in adf.test(ts_ng_diff): p-value smaller than printed p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  ts_ng_diff
## Dickey-Fuller = -6.3165, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
MannKendall(ts_ng_diff)
```

```
## tau = -0.00409, 2-sided pvalue =0.92755
```

```r
#ACF and PACF after two differencings
par(mfrow=c(1,2))
Acf(ts_ng_diff, lag=40, main="Differenced NG Generation")
Pacf(ts_ng_diff, lag=40, main="Differenced NG Generation")
```




```r
par(mfrow=c(1,1))

#Candidates
model_s1 <- Arima(ts_ng_gen, order=c(1,1,1), seasonal=c(1,1,1))
model_s2 <- Arima(ts_ng_gen, order=c(0,1,1), seasonal=c(0,1,1))
model_s3 <- Arima(ts_ng_gen, order=c(1,1,1), seasonal=c(0,1,1))
model_s4 <- Arima(ts_ng_gen, order=c(2,1,1), seasonal=c(0,1,1))

#AIC values
AIC(model_s1, model_s2, model_s3, model_s4)
```

```
##          df      AIC
## model_s1  5 4554.347
## model_s2  3 4565.282
## model_s3  4 4552.393
## model_s4  5 4553.673
```

```r
summary(model_s3)
```

```
## Series: ts_ng_gen
## ARIMA(1,1,1)(0,1,1)[12]
##
## Coefficients:
##          ar1      ma1     sma1
##       0.7323  -0.9819  -0.7017
## s.e.  0.0504   0.0183   0.0563
##
## sigma^2 = 27922085:  log likelihood = -2272.2
## AIC=4552.39   AICc=4552.57   BIC=4566.09
##
## Training set error measures:
##                    ME    RMSE      MAE        MPE    MAPE      MASE        ACF1
## Training set 388.6732 5104.96 3966.553 0.02702438 4.66492 0.4849429 -0.04631823
```

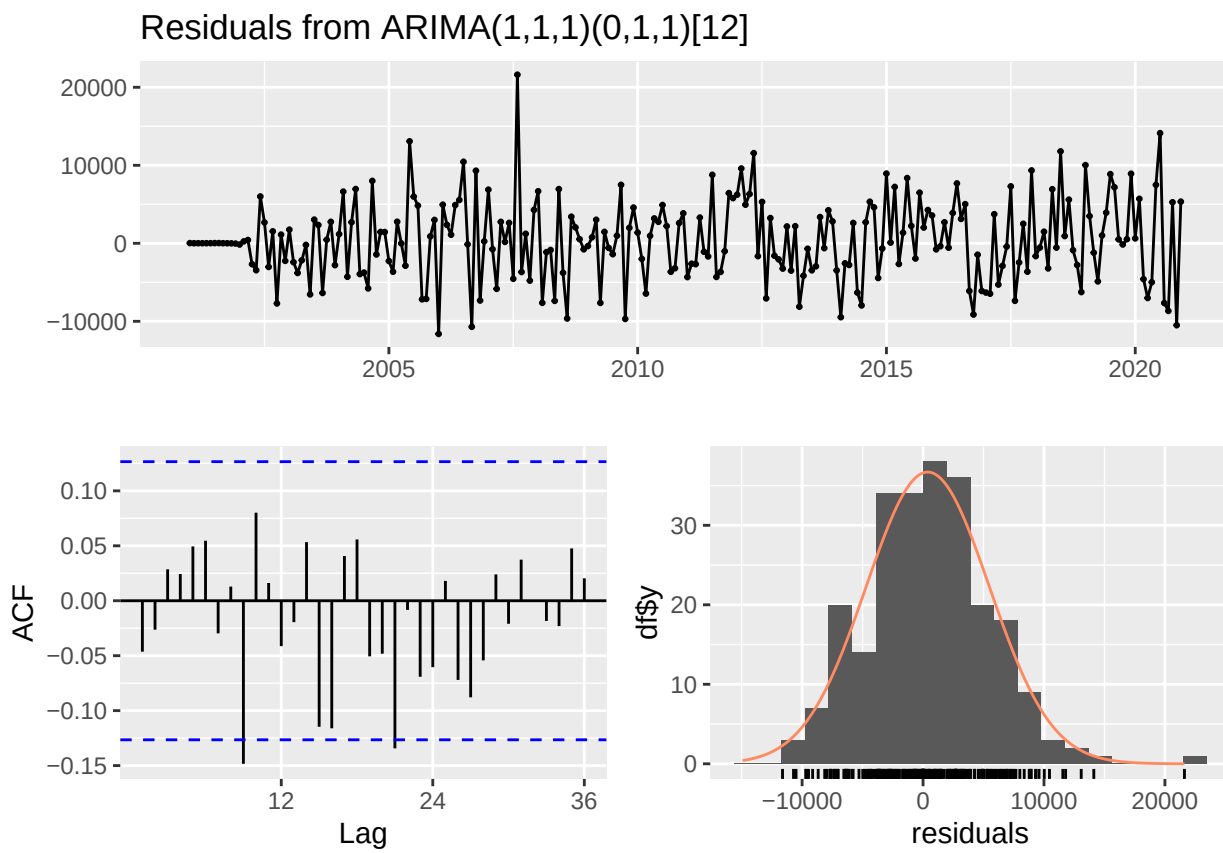
```r
cat("Model Coefficients:\n")
```

```
## Model Coefficients:
```

```r
print(model_s3$coef)
```

```
##        ar1        ma1       sma1
##  0.7323143 -0.9819269 -0.7016664
```

```r
#Residuals
checkresiduals(model_s3)
```

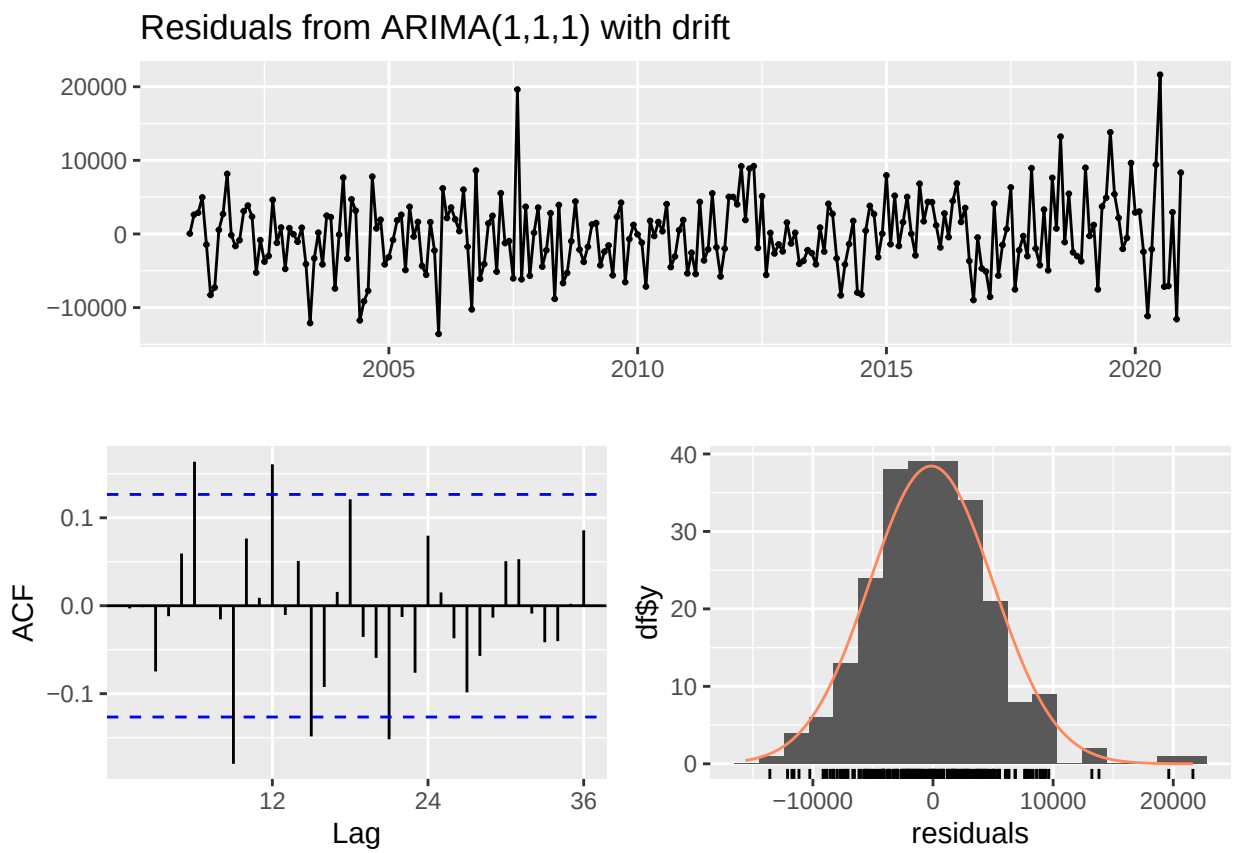



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(0,1,1)[12]
## Q* = 27.607, df = 21, p-value = 0.1516
##
## Model df: 3.   Total lags used: 24
```

Q8

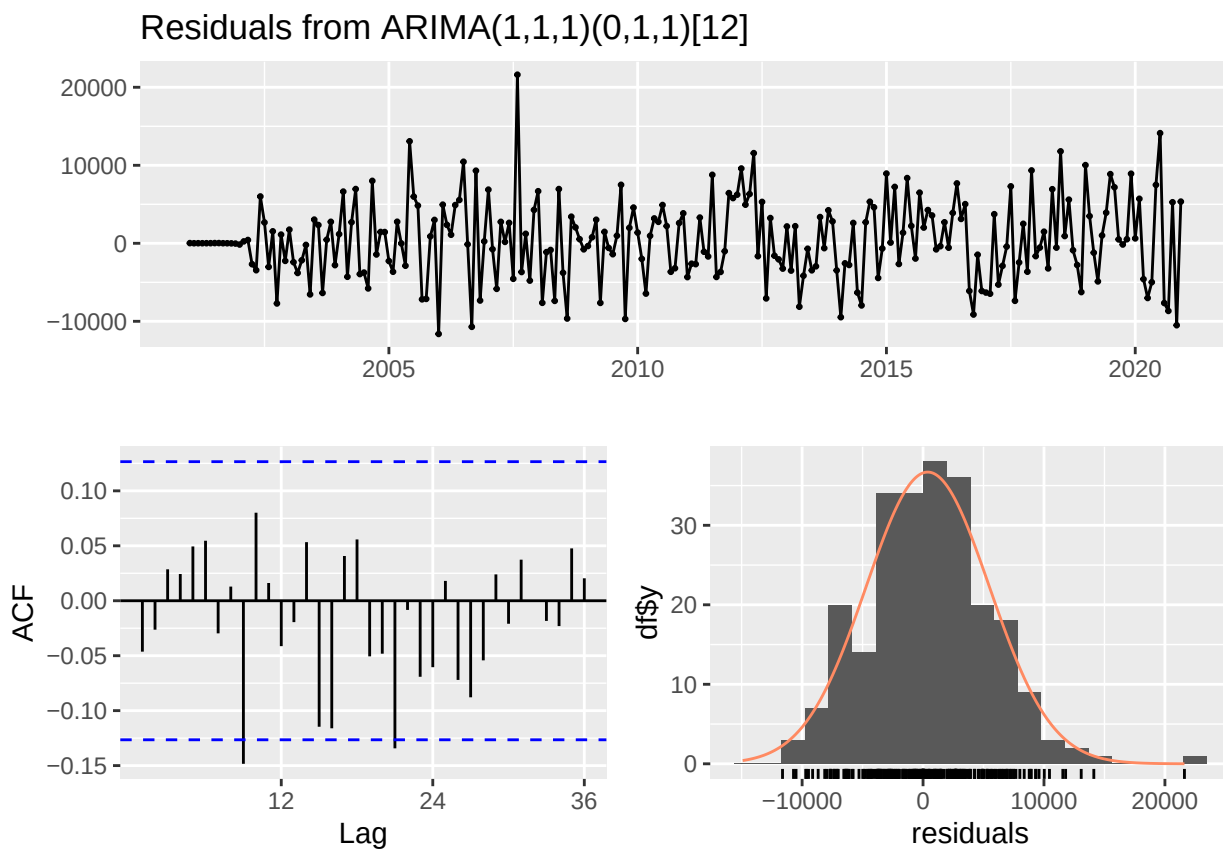
Compare the residual series for Q7 and Q6. Can you tell which ARIMA model is better representing the Natural Gas Series? Is that a fair comparison? Explain your response.
> Answer: The ARIMA(1,1,1)(0,1,1)12 is better representing the Natural Gas series than the ARIMA(1,1,1) from Q6. The residual ACF of the Q7 model shows no spikes.

```
checkresiduals(model_111_drift)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1) with drift
## Q* = 48.356, df = 22, p-value = 0.0009736
##
## Model df: 2.   Total lags used: 24
```

```r
checkresiduals(model_s3)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(0,1,1)[12]
## Q* = 27.607, df = 21, p-value = 0.1516
##
## Model df: 3.   Total lags used: 24
```

Checking your model with the auto.arima()

Please do not change your answers for Q4 and Q7 after you ran the *auto.arima*(). It is **ok** if you didn't get all orders correctly. You will not loose points for not having the same order as the *auto.arima*().

Q9

Use the *auto.arima*() command on the **deseasonalized series** to let R choose the model parameter for you. What's the order of the best ARIMA model? Does it match what you specified in Q4?

> Answer: The best ARIMA model by auto.arima is ARIMA(1,1,1), which matches with my answer in Q4.

```
auto.arima(deseason_ng)
```

```
## Series: deseason_ng
## ARIMA(1,1,1) with drift
##
## Coefficients:
##          ar1      ma1     drift
##       0.7065  -0.9795  359.5052
## s.e.  0.0633   0.0326   29.5277
##
## sigma^2 = 26980609:  log likelihood = -2383.11
## AIC=4774.21   AICc=4774.38   BIC=4788.12
```

Q10

Use the *auto.arima*() command on the **original series** to let R choose the model parameters for you. Does it match what you specified in Q7?

> Answer: The best model that R choose is ARIMA(1,0,0)(0,1,1)[12] with drift, which does not match with my result in Q7.

```r
auto.arima(ts_ng_gen)
```

```
## Series: ts_ng_gen
## ARIMA(1,0,0)(0,1,1)[12] with drift
##
## Coefficients:
##          ar1     sma1     drift
##       0.7416  -0.7026  358.7988
## s.e.  0.0442   0.0557   37.5875
##
## sigma^2 = 27569124:  log likelihood = -2279.54
## AIC=4567.08   AICc=4567.26   BIC=4580.8
```